

DeepCircuitX: A Comprehensive Repository-Level Dataset for RTL Code Understanding, Generation, and PPA Analysis

Zeju Li ^{1,6†}, Changran Xu ^{1,6†}, Zhengyuan Shi ^{1,6†}, Zedong Peng ^{2,6}, Yi Liu ^{1,6}, Yunhao Zhou ^{1,6}, Lingfeng Zhou ^{3,6}, Chengyu Ma ^{4,6}, Jianyuan Zhong ^{1,6}, Xi Wang ^{5,6}, Jieru Zhao ², Zhufei Chu ⁴, Xiaoyan Yang ³, Qiang Xu ^{1,6*}

¹Department of Computer Science and Engineering, The Chinese University of Hong Kong, Sha Tin, Hong Kong S.A.R.

²Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China

³School of Computer Science, Hangzhou Dianzi University, Hangzhou, China

⁴Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo, China

⁵School of Integrated Circuit, Southeast University, Nanjing, China

⁶National Technology Innovation Center for EDA, Nanjing, China

Abstract—This paper introduces DeepCircuitX, a comprehensive repository-level dataset designed to advance RTL (Register Transfer Level) code understanding, generation, and power-performance-area (PPA) analysis. Unlike existing datasets that are limited to either file-level RTL code or physical layout data, DeepCircuitX provides a holistic, multilevel resource that spans repository, file, module, and block-level RTL code. This structure enables more nuanced training and evaluation of large language models (LLMs) for RTL-specific tasks. DeepCircuitX is enriched with Chain of Thought (CoT) annotations, offering detailed descriptions of functionality and structure at multiple levels. These annotations enhance its utility for a wide range of tasks, including RTL code understanding, generation, and completion. Additionally, the dataset includes synthesized netlists and PPA metrics, facilitating early-stage design exploration and enabling accurate PPA prediction directly from RTL code. We demonstrate the dataset's effectiveness on various LLMs finetuned with our dataset and confirm the quality with human evaluations. Our results highlight DeepCircuitX as a critical resource for advancing RTL-focused machine learning applications in hardware design automation. Our data is available at <https://github.com/cure-lab/LCM-Datasetm>.

Index Terms—LLM, Code Understanding, Code Generation

I. INTRODUCTION

Register Transfer Level (RTL) modeling is a crucial step in the Electronic Design Automation (EDA) flow, where RTL code (e.g., Verilog and VHDL) represents hardware functionality, bridging design specifications and circuit implementations. Engineers then translate RTL into netlists, floorplans, and layouts, directly influencing circuit design quality. Integrating Artificial Intelligence (AI) into the EDA flow, particularly for RTL tasks, promises to enhance modern chip design and accelerate time-to-market. For instance, [1] employs large language models (LLMs) to understand and describe RTL designs in natural language to assist engineers. [2], [3] focus on generating RTL code from the design specification. Clearly, the effectiveness of deep learning models is heavily contingent upon the quality of the training data [4]. However, we observe that existing RTL datasets suffer from notable limitations, hindering the application of AI-based solutions in practical RTL modeling and verification.

One prominent limitation is the narrow scope of most datasets. Since the semiconductor ecosystem is far inferior to software openness, the accessible designs are limited. Many datasets either focus on a limited set of circuit types, often centering around a single design such as processors [5], [6], or indiscriminately collect all Verilog files without accounting for their correctness and distribution [3], [7]. This restricted diversity hampers the generalizability and effectiveness of these datasets.

Another critical limitation is the lack of focus on circuit implementation in RTL-based datasets. For instance, RTLLM [8] gathers RTL specifications and verifies the functional correctness of RTL code, while [4] ensures functionality when constructing and augmenting datasets. RTL-Repo [1] collects 1000+ RTL design projects and make each sample in the dataset contains the context of the entire repository to train LLM for RTL code understanding. However, these works do not consider or include circuit-level information, such as netlists and layouts, which are crucial in later stages of the EDA flow. As a result, these datasets overlook the correlation between RTL code and its corresponding circuit implementations. Consequently, training RTL models with the existing datasets that both guarantee functional correctness and optimize Power, Performance, and Area (PPA) remains a significant challenge.

To overcome these limitations, we present a multimodal and comprehensive repository-level circuit dataset for deep learning in EDA. Our dataset is meticulously curated according to the scope of the real-world chip and collect more than 4,000 circuit design projects from various data sources. Unlike previous datasets that provide verilog files only [3], [4], each data point in our dataset is structured as a complete repository and also splitted into files, modules and blocks for various scenarios. This multi-tiered structure enables training models at different scales, making it suitable for a variety of LLMs and models. Moreover, the repository-level data allows logic synthesis and physical design flows to convert RTL code into various circuit modalities, including Control/Data Flow Graph (CDFG) of RTL code, And-Invertor Graph (AIG) for [9], post-mapping netlist for [10], floorplaning and layout. Therefore, we can build the interactions between RTL designs and circuit implementations.

Furthermore, we meticulously label the circuits for unimodal RTL tasks and cross-modal PPA prediction to enhance accessibility of our dataset. We propose a Chain of Thought (CoT) [11] detailed annotation method to generate descriptions and comments for each of the four levels, namely, repo-level, file-level, module-level and block-level. By using GPT-4 [12] and Claude [13], we leverage annotations from higher levels to assist in annotating lower levels. Additionally, we generate question-answer pairs to help describe the functionality and key features of each code segment, enabling better training data for LLMs. Moreover, we provide the corresponding logic synthesis results for the cross-stage PPA prediction on RTL code.

With this dataset, we create a range of pre-training tasks, evaluation tasks, and benchmarks. One experiment focuses on RTL code understanding and generation with LLMs. We trained models including CodeLLama [14], CodeT5+ [15], CodeGen [16], and DeepSeek [17],

† Equal Contribution, * Corresponding author (qxu@cse.cuhk.edu.hk)

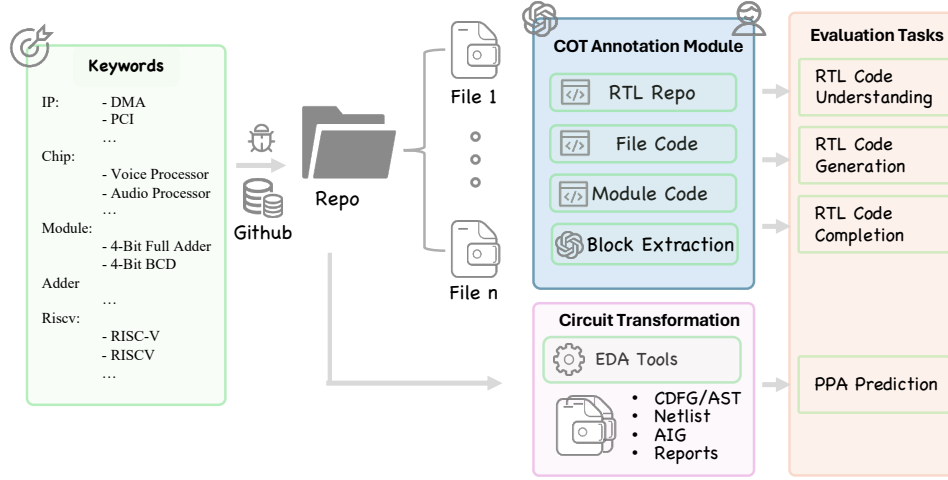


Fig. 1. Pipeline overview of the proposed framework, illustrating the key stages: data collection from GitHub using keywords, data annotation via chain-of-thought (COT), circuit transformation, and evaluation, including RTL code tasks for LLM and PPA prediction.

[18] at scales ranging from 220M to 16B parameters on our dataset. Another experiment evaluates PPA using learning-based prediction models [19]–[21], trained to assess area, power, and delay. The results reveal three key findings: 1) Fine-tuning LLMs on our dataset leads to significant performance improvements across all metrics, demonstrating the effectiveness of our dataset. 2) LLMs of different scales, such as CodeT5 220M, 7B, and 16B, show substantial gains after fine-tuning, indicating the adaptability of our dataset across varying model sizes. 3) PPA prediction results highlight the ongoing challenge of accurate early-stage PPA forecasting, providing valuable insights for future RTL and hardware design automation research. And the pipeline and framework are illustrated in Figure 1.

The main contributions of our work are summarized as follows:

- We propose a holistic dataset including over 4,000 repository-level RTL projects, covering chip-designs, IP-designs, module-designs designs, and incorporating a diverse range of functional and algorithmic keywords.
- Our dataset is organized into four levels, namely, repository, file, module, and block levels, allowing models to be trained at different scales and enabling broader applications in EDA tasks such as synthesis, netlist generation, PPA analysis, and layout.
- We propose a Chain of Thought (CoT) annotation method using GPT-4 and Claude to generate detailed comments, descriptions, and question-answer pairs, improving training data for LLMs on the tasks of RTL code understanding and generation.
- We create pre-training and evaluation benchmarks for LLMs on RTL tasks such as code understanding, completion, and neural network-based PPA prediction, demonstrating the effectiveness, adaptability and generalization capabilities of our dataset for both RTL code comprehension and EDA tasks.

II. RELATED WORK

A. Previous Dataset in EDA

AI-based methodologies excel in addressing classification, prediction, and optimization tasks, making them well-suited for chip design. Over the past decade, the integration of AI into Electronic Design Automation (EDA) has emerged as an attractive direction in the fields of chip design and semiconductor industry, which is well surveyed in [22], [23].

The cornerstone of training AI models lies in the quality of the dataset utilized. In the domain of EDA, existing datasets can be broadly categorized into two classes. Firstly, there are unimodal datasets or benchmarks such as ISCAS'89 [24], ITC'99 [25], IWLS'05 [26] and EPFL [27] benchmarks, which offer open-source circuit netlists primarily for front-end applications like logic synthesis and design for test. [28] presents a dataset of physical designs generated from the IWLS'05 benchmark circuit suite, utilizing the open-source 130nm Process Design Kit (PDK) by Skywater and the OpenROAD toolkit [29]. Despite these efforts, these datasets still lack in providing comprehensive insights into cross-stage EDA tasks, such as optimizing circuit designs at the front-end for improved PPA metrics during the back-end stage.

Secondly, other multimodal circuit datasets are generated using EDA tools. For instance, CircuitNet [5], [6] creates a dataset through logic synthesis and physical design, where the circuit designs are synthesized into gate-level netlists and transformed into layouts using commercial EDA tools. [6] expand this work by providing data for million-gate designs such as CPUs, GPUs, and AI chips, and using the 14nm FinFET technology node to capture the increased complexity of manufacturing and modeling. Nonetheless, the existing datasets encompass a limited range of circuit designs, thereby constraining the generalizability of subsequent model training efforts.

B. RTL-stage Understanding, Completion, and Generation

[3] collects approximately 50,000 open-source Verilog code samples and fine-tunes five pre-trained LLMs, with model sizes ranging from 345 million to 16 billion parameters. [30] introduces a comprehensive evaluation dataset consisting of 156 problems sourced from HDLBits and developed a benchmarking framework to automatically test the functional correctness of Verilog code completions. Similarly, [31] fine-tunes existing LLMs on Verilog datasets collected from GitHub and textbooks, evaluating the functional correctness of the generated code using a custom test suite. [4] designs a data augmentation framework for training Chip Design LLMs, enabling them to generate Verilog code, EDA scripts, and coordinate EDA workflows based on natural language design descriptions. They also benchmark their approach by fine-tuning Llama 2 models with 7 billion and 13 billion parameters. Lastly, [32] presents the MG-

Verilog dataset, an open-source dataset that meets essential criteria for high-quality hardware data, facilitating the effective use of LLMs in hardware design. However, these datasets focus solely on file-level RTL code [33], neglecting the comprehensive information contained within entire RTL project designs, modules, and code blocks. As a result, they fail to provide the multimodal data, such as netlists and PPA metrics, that can be obtained through synthesis. Moreover, existing studies primarily focus on RTL code generation, often overlooking the critical aspects of annotations, comments, and descriptions for RTL code. Hence, there has been little progress in advancing RTL code understanding tasks, particularly for LLMs.

III. DATASET

A. Data Preparation

To maximize the utility of our dataset and enable training and evaluation for LLMs, it is essential to clearly distinguish between chip-level, IP-level, and module-level RTL designs. The repositories of different levels differ in abstraction, scope, and their roles in hardware design automation. In detail, chip-level design encompasses the entire System-on-Chip (SoC) and processors, focusing on system-wide integration, interconnects, and global constraints like power, timing, and area. IP-level design addresses modular and reusable components, such as memory controllers, with a focus on standalone functionality and scalability. Module-level design targets subcomponents within IPs, such as arithmetic units or fundamental operators, emphasizing detailed functionality and local optimization. These levels differ in scope, with chip-level representing the holistic system, IP-level the building blocks, and module-level the internal details, enabling hierarchical abstraction and efficient hardware design.

Our dataset distinguishes itself by focusing on chip-level, IP-level, module-level RTL designs. We collect a list of 222 keywords representing these levels from sources like Alldatasheet [34] and other relevant websites. Examples include chip-level designs such as voice processors, audio processors, and video processors; IP-level designs like DMA, PCI, and true random number generators; and module-level designs including 4-bit binary full adders, arithmetic logic units, and multiplier-accumulators. We collect over 4,000 repository-level RTL projects, which encompass 140,000 RTL files across 77 functional categories, as shown in Table I. Moreover, the detailed data cases are illustrated in Figure 2.

To construct the RTL-language dataset, we organize the data into four distinct levels: repository, file, module, and block. We employ a Chain of Thought (CoT) approach for RTL code annotation, leveraging GPT-4 [12] and Claude [13] to generate detailed comments, descriptions, and question-answer pairs. This methodology enhances the training data for large language models (LLMs) in RTL code understanding and generation, with further details provided in Section III-B. Additionally, to develop the multimodal dataset, we synthesize the RTL projects to obtain netlists, power, performance, and area (PPA) metrics, as well as layout designs. Further details will be discussed in Section III-C2.

TABLE I

SUMMARY OF DATA ACROSS DIFFERENT LEVELS IN DEEPCIRCUITX. THE TABLE PRESENTS THE NUMBER OF FUNCTION CATEGORIES, REPOSITORIES, AND RTL FILES AT THE CHIP, IP, MODULE DESIGNS.

Level	Function Categories	Repo Number	RTL File Number
Chip Level	17	1002	54650
IP Level	3	1410	92467
Module Level	57	2383	38692

B. Language-RTL Code Dataset Construction

1) *Chain of Thought (CoT) for RTL Code Annotation*: The Chain of Thought (CoT) reasoning method, commonly used in deep learning and natural language processing, simulates human reasoning by guiding models through structured steps. We adopt this approach to create detailed annotations for Verilog code, ensuring a comprehensive understanding at different levels of the RTL design. The annotation process is carried out at three distinct levels: **module-level**, **block-level**, and **repository-level**.

a) *Module-Level Annotations*: To annotate RTL modules, we utilize a multi-round question-answering approach to extract key information from the Verilog code, focusing on aspects such as the module's name, input/output ports, and internal signals. This process begins with structured questions designed to clarify both what the module does (What) and how it achieves this functionality (How). The responses provide the foundational details about the module's components and behavior. Using this information, a detailed specification is created that summarizes the module's name, its purpose, the roles of the input and output ports, and an explanation of the internal signals. Additionally, the specification includes a breakdown of the functional blocks within the module. Finally, with this specification and the original code, ChatGPT generates a concise yet comprehensive module-level annotation, encapsulating both the functionality and the implementation details.

b) *Block-Level Annotations*: Block-level annotations offer detailed insights into the functionality of specific sections within an RTL module. Given the complexity of Verilog code, which often includes nested structures, segmenting it using regular expressions alone can be challenging. To address this, we leverage GPT-4 for handling these intricacies. The code is divided into distinct functional blocks, including constructs such as `always`, `initial`, `task`, `function`, `generate`, `assign`, and `final` blocks. While regular expressions are employed to improve segmentation accuracy for straightforward patterns, GPT-4 is crucial for managing more complex structures. Once the blocks are identified, each is annotated to describe both its purpose and how it functions, ensuring that the role of each block within the module is clearly defined, thereby enhancing the overall understanding of the module's functionality.

c) *Repo-Level Annotations*: At the repository level, we aim to provide an overarching understanding of the entire RTL project. This includes information about the structure, purpose, and interconnections of various modules and files within the repository. To achieve this, we gather the file structure information and combine it with module-level annotations to form a complete picture of the project. GPT-4 is then used to summarize the contents of the repository, producing a top-down view of how individual files and modules work together. This approach allows the repo-level annotation to reflect both functional and design-level details, capturing the broader goals of the RTL project.

TABLE II

SUMMARY OF ANNOTATED RTL CATEGORIES AT THE MODULE, BLOCK, AND REPOSITORY LEVELS IN DEEPCIRCUITX. THE TABLE HIGHLIGHTS THE NUMBER OF ANNOTATED MODULES, BLOCKS, AND REPOSITORIES FOR EACH RTL CATEGORY, INCLUDING CHIP, IP, AND MODULE.

RTL categories	Module-Level	Block-Level	Repo-Level
Chip	7587	36955	684
IP	12863	20101	183
Module	28901	-	1389

2) *Dataset for RTL Code Understanding, Completion and Generation*: In addition to annotations, we constructed a dataset that supports three distinct tasks related to RTL code: **understanding**, **completion**, and **generation**. Table III shows the data counts across RTL categories (IP, Module, and Chip) for each task.

TABLE III
DATA COUNTS FOR CODE GENERATION, CODE COMPLETION, AND COMMENT GENERATION TASKS ACROSS DIFFERENT CATEGORIES.

Tasks Dataset	IP	Module	Chip	All
RTL Understanding	6386	14499	5270	26155
RTL Completion	6178	14131	5134	25443
RTL Generation	6479	16511	5343	28333

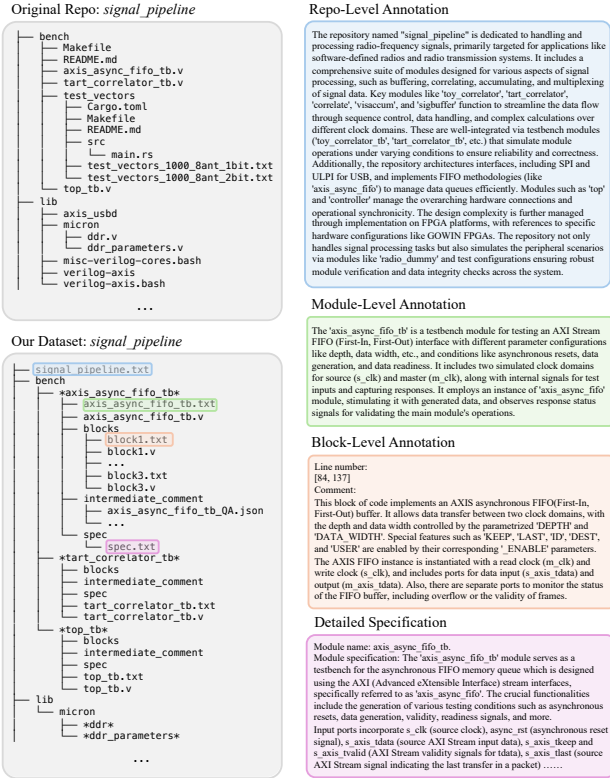


Fig. 2. Illustration of the dataset repository structure with multi-level annotations

a) *RTL Code Understanding*: This task evaluates the model's ability to interpret and describe RTL code. Given a module's RTL code as input, the model generates a detailed, concise description, covering key aspects such as the module's purpose, input/output signals, internal logic, and overall behavior. This task is crucial for assessing the model's ability to generate human-readable explanations for code analysis and documentation.

b) *RTL Code Completion*: In this task, the model is provided with a partial RTL code (typically the module header with input/output ports and parameters). The goal is for the model to complete the code by generating the missing internal logic, control structures, and signal definitions. This task mirrors autocompletion

functionality found in modern code editors and evaluates the model's ability to infer and generate code from context.

c) *RTL Code Generation*: In RTL code generation task, the model is tasked with producing a full implementation of RTL code based on a high-level description and specified input and output parameters. The goal is to generate a fully functional Verilog module that adheres to the provided specifications. This task assesses the model's ability to translate design requirements into precise RTL implementations, which is critical to automating the hardware design process.

C. Multimodal Transformation of RTL Code

1) *Graph-based Code Representation*: Abstract Syntax Tree (AST) serves as a tree-based representation to model the structure of programming code, extensively used in syntax analysis and compilation. In the context of Verilog code, each node within the AST denotes a variable or operator, and the edges formulate the relations between these nodes. Additionally, Control/Data Flow Graph (CDFG) [35] is another graph-based code representation to visualize how data moves between registers and how control signals dictate the operation of the design. CDFG proves advantages for modeling and verifying the functionality of RTL designs [36], [37]. We generate the AST and CDFG of RTL designs with open-source tools Yosys [38] and customized Pyverilog [39].

2) *Circuit Netlist Synthesis*: Our dataset contains RTL repositories with module invocations and complete Verilog files, enabling us to derive the circuit netlists via logic synthesis process. For each RTL repository, we employ the commercial tool Synopsys Design Compiler 2019.12 to transform HDL code into netlists. The RTL designs are mapped into several open-source technology libraries, including GlobalFoundries 180nm, skywater 130nm, ihp-sg 130nm, nangate 45nm and asap 7nm. Each RTL file is synthesized using both the compile and compile_ultra commands. By toggling the set max area 0 option, we obtain both the default netlist and a netlist optimized for maximum area constraints. We assess the internal, transition, and leakage power of each mapped netlist using PrimeTime (Synopsys, 2023.12). The critical path delay is reported by the Design Compiler.

Finally, we store the post-mapping netlist into both verilog format (.v files) and Standard Delay Format (SDF) [40] (.sdf files). Moreover, we record the logic synthesis reports (.rpt files) with maximum path delay, area and dynamic power for the following experiments. We also convert the post-mapping netlist into And-Inverter Graph (AIG) format, a prevalent and widely-adopted common format for netlist learning [41]–[44], using abc [45] tool for further investigation. The multi-modal data cases are illustrated in Figure 3.

IV. BENCHMARK AND EXPERIMENTS

A. overview

To evaluate our multi-modal dataset DeepCircuitX and establish benchmarks, we design experiments of human evaluation in Section IV-B, RTL code tasks in Section IV-C and Section IV-D, and PPA prediction experiments in Section IV-E.

Specially, in Section IV-C and Section IV-D, we establish benchmarks for LLMs tailored to RTL tasks, utilizing open-source models such as CodeLlama [14], CodeT5+ [15], deepseek-coder [46] and CodeGen [16]. Our annotated data is employed to fine-tune these models for RTL code understanding, completion, and generation tasks. Here is an concise iintroduction of the based-LLMs we select:

- CodeLlama is fine-tuned Llama on a diverse corpus of code from various programming languages, enabling it to understand syntax, semantics, and the context of code snippets.

- CodeT5+ is an enhanced version of the original CodeT5 model, specifically optimized for code understanding and generation tasks. Building on the strengths of its predecessor.
- CodeGen is trained on GPT-3 [47] by a diverse range of programming languages and employs advanced techniques to understand and generate code efficiently.
- DeepSeek-V2-lite [17] has broader applicability on general tasks of natural language and DeepSeek-Coder-V2-lite [18] focuses on code-related tasks, offering fine-tuned capabilities for high-quality code generation and synthesis.

B. Human evaluation

To evaluate the effectiveness of our generating comments approach and the quality of the annotations, we conduct a series of evaluation criteria and metrics involving human reviews by independent experienced engineers. We employ the following metrics: Accuracy, Completeness and Understandable Clarity. The detailed grading criteria is introduced in the Table IV.

TABLE IV
GRADING CRITERIA FOR ANNOTATION QUALITY.

Grade	Accuracy	Completeness	Understandable Clarity
4	Completely Accurate	Fully Complete	Clear and Concise
3	Partially Accurate	Mostly Complete	Relatively Clear
2	Not Quite Accurate	Not Quite Complete	Vague
1	Completely Incorrect	Incomplete	Inaccurate

Engineers are tasked with analyzing both the RTL code and the accompanying annotations based on specific criteria, ensuring a thorough evaluation of the provided information. Subsequently, they assign a grade to each metric on a scale from 1 to 4, with 4 indicating the highest quality and 1 indicating the lowest. To ensure fairness, each generated text is reviewed by 5 individuals, and the average score is recorded in Table V. We observe that the code annotations in DeepCircuitX exhibit high quality, with all metrics scoring above 3.5 out of 4.

TABLE V
HUMAN EVALUATION GRADING OF REPO-LEVEL ANNOTATION AND MODULE-LEVEL ANNOTATION, WE USE THE METRICS OF ACCURACY, COMPLETENESS, UNDERSTANDABLE CLARITY TO EVALUATE THE QUALITY OF OUR ANNOATION.

Metrics	Repo Annotation	Module Annotation
Accuracy	3.74/4	3.5/4
Completeness	3.79/4	3.78/4
Understandable Clarity	3.84/4	3.76/4

C. RTL Code Understanding

1) *Evaluation Metrics:* BLEU measures n-gram overlap (1 to 4) between generated text and reference translations, incorporating a brevity penalty. METEOR accounts for synonymy and stemming, calculating a harmonic mean of precision and recall. ROUGE focuses on summarization, with variants like ROUGE-N for n-gram overlap and ROUGE-L for the longest common subsequence, emphasizing recall to assess content relevance.

2) *Analysis:* The experimental results, shown in Table VI, demonstrate the performance of various base Large Language Models (LLMs) and fine-tuned LLMs on our RTL code understanding benchmark, using evaluation metrics BLEU-4, METEOR, ROUGE-1, ROUGE-2, and ROUGE-L, which offer valuable insights into the

quality of generated RTL code in terms of surface-level linguistic similarity.

Initially, the original versions of the LLMs, such as CodeLlama, CodeT5+, CodeGen2, and DeepSeek, exhibit relatively low performance across most metrics. For example, CodeLlama (original) achieves a BLEU-4 score of 0.0828, while CodeGen2.5 (original) shows moderate improvement with a BLEU-4 score of 0.1060. Among the original models, DeepSeek-Coder-V2-lite (original) stands out, significantly outperforming others with a BLEU-4 score of 2.2387 and a ROUGE-1 score of 30.3208, indicating its strong baseline performance even before fine-tuning.

After fine-tuning on our dataset, every large model demonstrates significantly better performance across BLEU-4, METEOR, ROUGE-1, ROUGE-2, and ROUGE-L metrics compared to their original, non-fine-tuned counterparts. This highlights the effectiveness of our dataset. Moreover, models of various sizes, such as the 220M CodeT5, as well as larger 7B and 16B models, all show substantial improvements after fine-tuning. This indicates that our dataset is well-suited for models of different scales, providing strong adaptability and generalization.

Overall, the fine-tuned LLMs significantly outperform their original counterparts, indicating the importance of fine-tuning on domain-specific datasets like ours.

TABLE VI
THE RESULTS OF OUR BASE-LLMS AND FINE-TUNED LLMs BY OUR TRAINING DATASET ON OUR RTL CODE UNDERSTANDING BENCHMARK.

Based LLM	BLEU-4	METEOR	ROUGE-1	ROUGE-2	ROUGE-L
CodeLlama (original)	0.0828	5.9414	11.0046	0.3139	0.2581
CodeT5+ 220m-bimodal (original)	0.1410	4.3277	10.9925	0.5076	9.3043
CodeGen2 (original)	0.1082	3.7890	8.0311	0.1173	7.2161
CodeGen2.5 (original)	0.1060	4.8271	9.4001	0.3698	8.5856
DeepSeek-Coder-V2-lite (original)	2.2387	24.3311	30.3208	6.3163	27.4282
DeepSeek-V2-lite (original)	0.2311	3.3951	7.3720	0.2646	6.3360
CodeLlama (7b)	0.8619	18.2621	25.0461	6.4493	22.8182
CodeT5+ (220m-bimodal)	4.9067	23.5043	34.8671	9.9023	32.1642
CodeGen2 (1b)	7.7605	27.8049	37.2150	13.1385	34.0647
CodeGen2.5 (7b)	13.6858	34.7494	43.5244	18.5249	40.2223
DeepSeek-Coder-V2-lite (16b)	11.9180	33.5011	41.8527	17.2014	38.0473
DeepSeek-V2-lite (16b)	13.6972	39.5962	43.3732	19.0589	39.5962

D. RTL Code Completion and Generation

1) *Evaluation Metrics:* In the realm of RTL code completion and generation, the evaluation of model performance is critical to advancing intelligent programming tools. The Pass@k metric serves as a pivotal measure in this domain, quantifying the accuracy of code generation models by assessing their ability to produce valid solutions within the top-k predictions. Specifically, Pass@k evaluates whether the correct code snippet appears among the model's top k outputs, thereby providing insights into both the effectiveness and reliability of the model's predictions.

2) *Analysis:* Table VII compares the performance of both original and fine-tuned LLMs on RTL code completion and generation tasks, focusing on Pass@1 and Pass@5 on two evaluation benchmarks, RTLLM [8] and VerilogEval [30]. We choose the baseline LLMs as original versions of CodeLlama, CodeT5+, CodeGen2, and CodeGen2.5 exhibit negligible performance, with most Pass@K scores at 0% or near 0%.

Notably, every model fine-tuned with our dataset significantly outperforms its original, non-fine-tuned counterpart, demonstrating the effectiveness of our data. Additionally, for models of different scales, such as the 220M CodeT5 and 7B models, the results after fine-tuning show substantial improvements. This highlights the adaptability and generalization capability of our dataset across various model sizes.

Moreover, we include CodeV (QW-7B) [48] as an additional baseline, which achieves 14.80% Pass@1 and Pass@5 on RTLLM, and 4.5% on VerilogEval. Although CodeV has undergone prior fine-tuning for general-purpose code generation, its performance remains lower than our fine-tuned CodeGen2.5 (7B).

These findings highlight the effectiveness of our dataset in enhancing LLMs' capability to generate syntactically and functionally accurate RTL code.

TABLE VII

PASS@K RESULTS FOR RTL CODE COMPLETION AND GENERATION ACROSS VARIOUS LLMs. THE RESULT WITH ARE EVALUATED USING OUR CUSTOM IMPLEMENTATION WITH OUR EXPERIMENTAL SETTINGS.

Based LLM	Pass@1		Pass@5	
	RTLLM [8]	VerilogEval [30]	RTLLM	VerilogEval
CodeLLama (original)	0	0.64%	0	0
CodeT5+ (original)	0	0	0	0
CodeGen2 (original)	0	0	0	0.64%
CodeGen2.5 (original)	17.24%	23.08%	17.24%	24.36%
CodeLLama (7b)	6.90%	1.92%	6.90%	6.41%
CodeT5+ (220m-bimodal)	3.45%	4.49%	3.45%	4.49%
CodeGen2 (1b)	13.79%	10.90%	13.79%	10.90%
CodeV (QW 7b) [48]	14.80%*	4.5%*	14.80%*	4.5%*
CodeGen2.5 (7b)	24.14%	24.36%	24.14%	25%

E. PPA Prediction

1) *Overview*: Optimizing Power, Performance, and Area (PPA) stands out as a primary objective in the circuit design process. Estimating PPA metrics at an early stage not only boosts design efficiency but also empowers a more agile response to evolving design requirements and constraints. Unlike previous RTL datasets that solely gather Verilog files, our datasets encompass entire repositories, allowing us to obtain post-mapping netlists and logic synthesis reports by EDA tools. Consequently, we can parse the PPA metrics from the logic synthesis reports and predict them for the corresponding RTL designs at an early stage. In this subsection, we formulate the PPA prediction task within our dataset and assess the effectiveness of current learning-based prediction models [19]–[21].

2) *Evaluation Metrics*: We regard the circuit characteristics of the post-mapping netlist produced by the Design Compiler tool with default settings as the ground truth for PPA metrics. Specifically, we define the dynamic power under random workloads as the **Power** metric, the maximum path delay as the **Delay** metric and the total cell area as the **Area** metric.

We use Mean Absolute Error Percentage (MAPE) and Root Relative Square Error (RRSE) to evaluate the prediction accuracy of PPA. It should be denoted that the PPA predictor trained on the datasets generated with a certain technology library cannot be generalized to another, as the PPA metrics of netlists are strongly linked to the technology library. Therefore, we choose the netlists and logic synthesis reports with a specific library, which keeps consistency with the settings in [19]–[21]. We opt for skywater 130nm library in the following experiments.

3) *Analysis*: We collect 146 RTL designs for training and 10 designs for testing, with these designs containing more than 10k cells after logic synthesis and closely resembling practical designs. To explore the data scalability of PPA prediction models, we sample the training dataset size to 10%, 50% and 100%. It should be denoted that we exclude the model [20] for the delay prediction as it cannot support this task. According to the prediction results shown in Table VIII,

TABLE VIII
PPA PREDICTION RESULTS OF LEARNING-BASED MODELS (THE LOWER, THE BETTER).

Model	Data	Area		Power		Delay	
		MAPE	RRSE	MAPE	RRSE	MAPE	RRSE
SNS [19]	10%	1.9379	1.4913	1.9045	1.1362	49.3638	6.6901
	50%	0.7811	1.9041	0.8425	1.1759	47.7314	2.8816
	100%	0.6564	1.7197	0.7549	1.1740	4.7392	2.3647
[20]	10%	1.9066	1.0802	10.2783	14.7510	-	-
	50%	0.7954	0.6133	0.7478	0.3539	-	-
	100%	0.5996	0.5074	0.7343	0.6558	-	-
MasterRTL [21]	10%	1.3405	1.1197	2.3253	29.7767	80.7779	8.5936
	50%	0.5640	0.9982	1.7237	29.8749	24.7716	6.8725
	100%	0.3303	0.7605	0.6501	2.2541	3.4769	4.2863

we conclude three observations. Firstly, the accuracy of predictions improves as the training data volume increases. For instance, the MAPE of area prediction is 4.3201 with 10% data and decreases to 0.3303 with 100% data. Secondly, PPA predictors demonstrate weaker performance in delay prediction. For example, both models exhibit unsatisfactory performance, where [19] shows 4.7392 MAPE and [21] has 3.4769 MAPE. This suggests that estimating timing characteristics in the early stages using path features on RTL-based graphs is still difficult, as logic synthesis tools heavily optimize logic to minimize maximum path delays. Thirdly, in comparison to the originally reported performance on simple benchmarks, these models exhibit diminished performance on designs more than 10k cells in our dataset. Therefore, how to accurately predict PPA of practical designs remains an opening question, necessitating further exploration in the EDA community.

V. CONCLUSION

In this paper, we introduce **DeepCircuitX**, a multimodal and comprehensive repository-level dataset designed to advance RTL code understanding, generation, and PPA analysis in hardware design automation. By offering a holistic resource that spans repository, file, module, and block-level RTL code, DeepCircuitX enables large language models to better tackle the complexities of hardware design. The integration of Chain of Thought annotations further enhances the dataset's value by providing detailed insights into functionality and structure, thereby improving model training and performance across a variety of RTL tasks. Our experiments demonstrate that models trained on DeepCircuitX significantly outperform existing methods in tasks like code understanding, generation, and completion, as well as in RTL-to-PPA prediction. The inclusion of synthesized netlists and PPA metrics opens up new avenues for early-stage design exploration, offering a practical tool for both researchers and practitioners in EDA. As a result, DeepCircuitX establishes new benchmarks for RTL tasks, setting a foundation for future innovations in hardware design automation and demonstrating the potential of LLMs to transform this critical domain.

VI. ACKNOWLEDGMENTS

This work was supported in part by the General Research Fund of the Hong Kong Research Grants Council (RGC) under Grant No. 14212422 and 14202824, in part by Huawei Technologies Co. Ltd. under grant No. N2-2c-TH2420350, and in part by National Technology Innovation Center for EDA, China.

REFERENCES

- [1] A. Allam and M. Shalan, "Rtl-repo: A benchmark for evaluating llms on large-scale rtl design projects," *arXiv preprint arXiv:2405.17378*, 2024.
- [2] F. Cui, C. Yin, K. Zhou, Y. Xiao, G. Sun, Q. Xu, Q. Guo, D. Song, D. Lin, X. Zhang *et al.*, "Origen: Enhancing rtl code generation with code-to-code augmentation and self-reflection," *arXiv preprint arXiv:2407.16237*, 2024.
- [3] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated verilog rtl code generation," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–6.
- [4] K. Chang, K. Wang, N. Yang, Y. Wang, D. Jin, W. Zhu, Z. Chen, C. Li, H. Yan, Y. Zhou *et al.*, "Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework," *arXiv preprint arXiv:2403.11202*, 2024.
- [5] Z. Chai, Y. Zhao, Y. Lin, W. Liu, R. Wang, and R. Huang, "Circuitnet: An open-source dataset for machine learning applications in electronic design automation (eda)," *arXiv preprint arXiv:2208.01040*, 2022.
- [6] X. Jiang, Y. Zhao, Y. Lin, R. Wang, R. Huang *et al.*, "Circuitnet 2.0: An advanced dataset for promoting machine learning innovations in realistic chip design environment," in *The Twelfth International Conference on Learning Representations*, 2024.
- [7] B.-Y. Wu, U. Sharma, S. R. D. Kankipati, A. Yadav, B. K. George, S. R. Guntupalli, A. Rovinski, and V. A. Chhabria, "Eda corpus: A large language model dataset for enhanced interaction with openroad," 2024. [Online]. Available: <https://arxiv.org/abs/2405.06676>
- [8] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "Rtllm: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.
- [9] S. Khan, Z. Shi, M. Li, and Q. Xu, "Deepseq: Deep sequential circuit learning," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–2.
- [10] Z. Zheng, S. Huang, J. Zhong, Z. Shi, G. Dai, N. Xu, and Q. Xu, "Deepgate4: Efficient and effective representation learning for circuit design at scale," *arXiv preprint arXiv:2502.01681*, 2025.
- [11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [12] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [13] Anthropic, "Claude 3.5 sonnet model card addendum," Anthropic, Technical Report, 2024, accessed: [Insert access date here]. [Online]. Available: https://www-cdn.anthropic.com/fed9cc193a14b84131812372d8d5857f8f304c52/Model_Card_Claude_3_Addendum.pdf
- [14] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [15] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "Codet5+: Open code large language models for code understanding and generation," *arXiv preprint arXiv:2305.07922*, 2023.
- [16] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.
- [17] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.
- [18] Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma *et al.*, "Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence," *arXiv preprint arXiv:2406.11931*, 2024.
- [19] C. Xu, C. Kjellqvist, and L. W. Wills, "Sns's not a synthesizer: a deep-learning-based synthesis predictor," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 847–859.
- [20] P. Sengupta, A. Tyagi, Y. Chen, and J. Hu, "How good is your verilog rtl code? a quick answer from machine learning," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.
- [21] W. Fang, Y. Lu, S. Liu, Q. Zhang, C. Xu, L. W. Wills, H. Zhang, and Z. Xie, "Masterrtl: A pre-synthesis ppa estimation framework for any rtl design," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [22] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, J. Wu, Y. Xu, H. Zhang, K. Zhong *et al.*, "Machine learning for electronic design automation: A survey," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 26, no. 5, pp. 1–46, 2021.
- [23] L. Chen, Y. Chen, Z. Chu, W. Fang, T.-Y. Ho, Y. Huang, S. Khan, M. Li, X. Li, Y. Liang *et al.*, "The dawn of ai-native eda: Promises and challenges of large circuit models," *arXiv preprint arXiv:2403.07257*, 2024.
- [24] F. Brglez, D. Bryan, and K. Kozminski, "Notes on the iscas'89 benchmark circuits," *North-Carolina State University*, 1989.
- [25] S. Davidson, "Characteristics of the itc'99 benchmark circuits," in *ITSW*, 1999.
- [26] C. Albrecht, "Iwls 2005 benchmarks," in *IWLS*, 2005.
- [27] L. Amarú, P.-E. Gaillardon, and G. De Micheli, "The epfl combinational benchmark suite," in *IWLS*, no. CONF, 2015.
- [28] P. Shrestha, A. Aversa, S. Phatharodom, and I. Savidis, "Eda-schema: A graph datamodel schema and open dataset for digital design automation," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, 2024, pp. 69–77.
- [29] T. Ajayi and D. Blaauw, "Openroad: Toward a self-driving, open-source digital layout implementation tool chain," in *Proceedings of Government Microcircuit Applications and Critical Technology Conference*, 2019.
- [30] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Verilogval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.
- [31] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.
- [32] Y. Zhang, Z. Yu, Y. Fu, C. Wan *et al.*, "Mg-verilog: Multi-grained dataset towards enhanced llm-assisted verilog generation," *arXiv preprint arXiv:2407.01910*, 2024.
- [33] Y. Liu, C. Xu, Y. Zhou, Z. Li, and Q. Xu, "Deeprtl: Bridging verilog understanding and generation with a unified representation model," *arXiv preprint arXiv:2502.15832*, 2025.
- [34] Alldatasheet, 2003. [Online]. Available: <https://www.alldatasheet.com/>
- [35] A. Orailoglu and D. D. Gajski, "Flow graph representation," in *Proceedings of the 23rd acm/ieee design automation conference*, 1986, pp. 503–509.
- [36] P. Coussy, D. D. Gajski, M. Meredith, and A. Takach, "An introduction to high-level synthesis," *IEEE Design & Test of Computers*, vol. 26, no. 4, pp. 8–17, 2009.
- [37] S. Vasudevan, W. J. Jiang, D. Bieber, R. Singh, C. R. Ho, C. Sutton *et al.*, "Learning semantic representations to verify hardware designs," *Advances in Neural Information Processing Systems*, vol. 34, pp. 23 491–23 504, 2021.
- [38] C. Wolf, "Yosys open synthesis suite," 2016.
- [39] S. Takamaeda-Yamazaki, "Pyverilog: A python-based hardware design processing toolkit for verilog hdl," in *Applied Reconfigurable Computing: 11th International Symposium, ARC 2015, Bochum, Germany, April 13-17, 2015, Proceedings 11*. Springer, 2015, pp. 451–460.
- [40] V. Sagdeo, "Standard delay format," *The Complete Verilog Book*, pp. 321–363, 1998.
- [41] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu, "Deepgate: Learning neural representations of logic gates," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 667–672.
- [42] Z. Shi, H. Pan, S. Khan, M. Li, Y. Liu, J. Huang, H.-L. Zhen, M. Yuan, Z. Chu, and Q. Xu, "Deepgate2: Functionality-aware circuit representation learning," in *2023 IEEE/ACM International Conference on Computer Aided Design*. IEEE, 2023.
- [43] Z. Shi, Z. Zheng, S. Khan, J. Zhong, M. Li, and Q. Xu, "Deepgate3: Towards scalable circuit representation learning," *arXiv preprint arXiv:2407.11095*, 2024.
- [44] C. Deng, Z. Yue, C. Yu, G. Sarar, R. Carey, R. Jain, and Z. Zhang, "Less is more: Hop-wise graph attention for scalable and generalizable learning on circuits," *arXiv preprint arXiv:2403.01317*, 2024.
- [45] R. Brayton and A. Mishchenko, "Abc: An academic industrial-strength verification tool," in *CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings 22*. Springer, 2010, pp. 24–40.
- [46] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

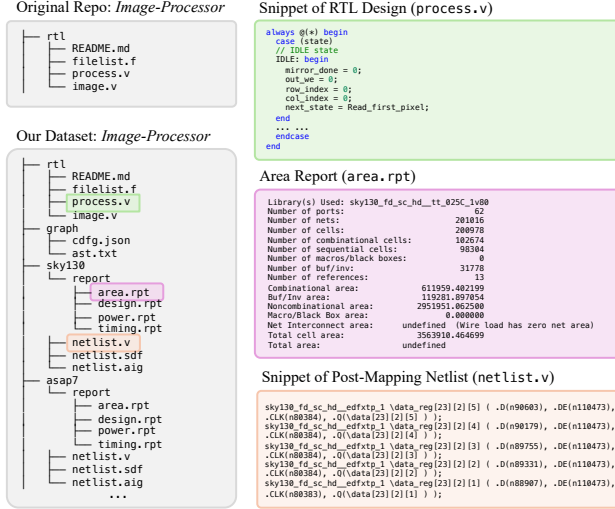


Fig. 3. Illustration of the dataset repository structure with Circuit Transformation

- [47] L. Floridi and M. Chiriatti, “Gpt-3: Its nature, scope, limits, and consequences,” *Minds and Machines*, vol. 30, pp. 681–694, 2020.
- [48] Y. Zhao, D. Huang, C. Li, P. Jin, Z. Nan, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang *et al.*, “Codev: Empowering llms for verilog generation through multi-level summarization,” *arXiv preprint arXiv:2407.10424*, 2024.

APPENDIX

1. The grading criteria of **Accuracy**

- 4: Completely Accurate: The annotation accurately captures all functionalities, I/O signals, and operational logic without errors.
- 3: Partially Accurate: The annotation contains some inaccuracies in a limited number of functionalities, I/O signals, or operational logic descriptions.
- 2: Not Quite Accurate: The annotation has significant inaccuracies affecting a majority of functionalities, I/O signals, or operational logic descriptions.
- 1: Completely Incorrect: The annotation fails to accurately describe the functionalities, I/O signals, or operational logic.

2. The grading criteria of **Completeness**

- 4: Fully Complete: The annotation provides a comprehensive explanation of overall functionality, I/O signals, and operational logic.
- 3: Mostly Complete: The annotation is largely complete but misses minor details regarding overall functionality, I/O signals, or operational logic.

- 2: Not Quite Complete: The annotation is partially complete but misses many details regarding overall functionality, I/O signals, or operational logic.
 - 1: Incomplete: The annotation lacks significant portions of the overall functionality, I/O signals, or operational logic.
3. The grading criteria of **Understandable Clarity**
- 4: Clear and Concise: The annotation is articulated clearly with minimal redundancy.
 - 3: Relatively Clear: The annotation is generally clear but contains some redundant information.
 - 2: Vague: The annotation is unclear and lacks precision.
 - 1: Inaccurate or Incomplete: The annotation is both inaccurate and incomplete.

As illustrated in Figure 2, our dataset preserves the original file structure of the repository while extracting repository-level annotations and providing multi-level annotations for each Verilog file in the original repository. For each Verilog file, all annotation information is stored in a folder named “*<original file name>*”, which contains:

- <original file name>.txt: A text file containing the module-level annotations;
- <original file name>.v: The original Verilog file;
- blocks: A folder that contains the original code for each block from the Verilog file (block{i}.v) along with the corresponding block-level annotations (block{i}.txt). Each annotation includes the line numbers indicating the block’s position in the original code and a comment describing its functionality;
- intermediate_comment: A folder containing intermediate results generated by the LLM during the Chain of Thought (CoT) process;
- spec: A folder containing a file named spec.txt, which provides a detailed specification of the module.

When we construct the agent of module-level RTL code annotation, each round focuses on different aspects of the code:

Round 1:

- What is the name of the module in the code?
- Provide a brief explanation of the module’s functionality.

Round 2:

- What are the input and output ports of the module?
- Provide a simple explanation of each input and output port, and their respective roles within the module.

Round 3:

- What are the internal signals used in the module?
- Provide a brief explanation of each internal signal and its role in the module.